Consensus, Replication, and Fault-tolerance in Distributed Systems

Doug Terry
LinkedIn

November 2025

Part 1: Agreement aka Consensus

Agreement Protocols

Types of agreement?

- Clock synchronization
- Leader election
- Consensus
- Byzantine agreement
- Atomic multicast
- Replication
- Transaction commit

Agreement Protocols

Types of agreement?

- Clock synchronization
- Leader election
- ☑ Consensus
- Byzantine agreement
- Atomic multicast
- ☑ Replication
- ☑ Transaction commit

Distributed Consensus

- Any node may propose value
- Goals:
  - *Agreement*: if a value is chosen, all nodes agree on same value
  - *Non-triviality*: value chosen is one that is proposed
  - *Termination*: nodes eventually choose a value; non-faulty nodes eventually learn of the chosen value

CAP Theorem

It is only possible to provide two of:

1. **Consistency** (C)
2. **Availability** (A)
3. **Partition tolerance** (P)

Impossibility of Consensus

Fischer, Lynch, Paterson (FLP) result:

- Consensus not solvable if:
  - Asynchronous message passing
  - Nodes can fail by crashing
- Since protocol may not terminate
- Problem: cannot distinguish between failed, slow, or partitioned nodes
- But still can devise practical protocols…

Paxos Consensus Protocol

- First proposed by Lamport in 1989, but not understood until many years later
- Requires a majority of non-failed nodes to accept proposals
- Guarantees agreement, but not liveness
  - though works well enough in practice

Paxos Consensus Protocol

Assumptions:

- Fail-stop (i.e. non-Byzantine) failures
  - Nodes can crash and restart
- Arbitrary message delays, reordering, duplication, and drops
- Nodes have stable storage
- Number of nodes (i.e. majority) is known

Paxos Consensus Protocol

Three roles:

- *Proposers* make proposals
- *Acceptors* vote on proposals
- *Learners* discover the outcome

Paxos Consensus Protocol

- *Proposers* make proposals
  - each proposal has a unique number
  - different proposals maybe made concurrently
  - newer proposals are assigned higher numbers
  - proposals may be abandoned
    - e.g. if don't receive timely responses
    - e.g. if another leader makes a higher proposal

Paxos Consensus Protocol

- *Acceptors* vote on proposals
  - single acceptor is easy (can simply accept first proposal received), but not fault-tolerant
  - decision made when a majority accept
  - can accept multiple proposals
    - i.e. retract old votes that cannot win
  - promises never to accept lower-numbered proposals after replying to a higher one

Paxos Consensus Protocol

- *Learners* discover the outcome
  - easy since all nodes are trustworthy
  - acceptors could broadcast accepted values, but that would generate lots of traffic
  - could learn from leader when proposal accepted, but leader could fail
  - if in doubt, propose a value and see what gets accepted

Paxos Consensus Protocol

- Proposer in Phase I:

  v = value to propose

  n = new proposal number

  broadcast Prepare(n) to acceptors

  receive Promise(n, <num, value>) from a majority

  choice = value with highest number

Paxos Consensus Protocol

- Proposer in Phase II:

  if choice = NIL then choice = v

  broadcast Accept(n, choice) to acceptors

  receive Accepted from a majority

  choice is the consensus value

- Proposer in Phase III:

  broadcast Chosen(choice) to learners

Paxos Consensus Protocol

- Acceptor initially:

  accepted = <0, NIL>

  max = 0

- Acceptor on receipt of Prepare(n):

  if n > max

    max = n

    return Promise(n, accepted)

  else return Reject(n)

Paxos Consensus Protocol

- Acceptor on receipt of Accept(n, v):

  if n >= max

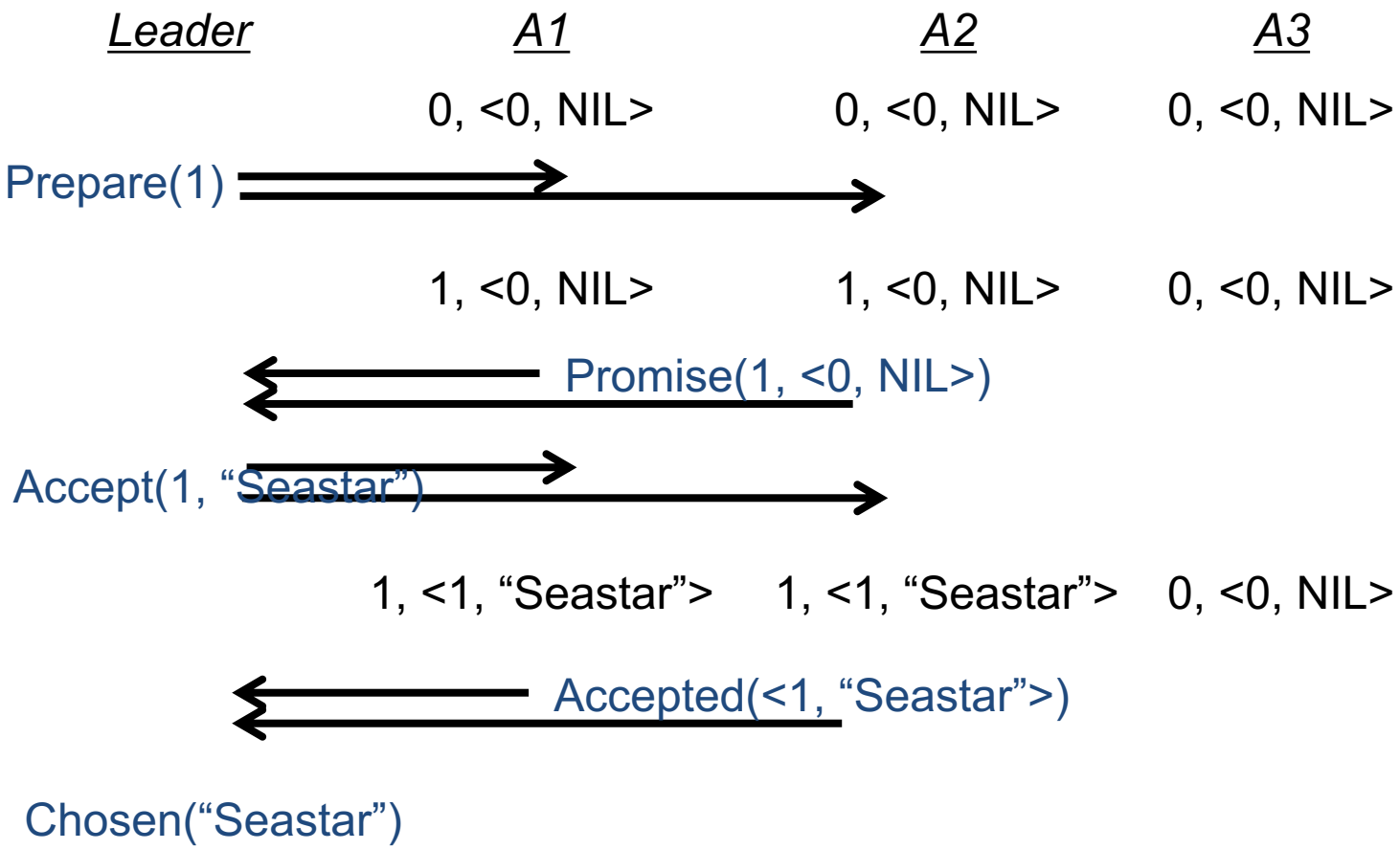
    max =n

    accepted = <n, v> -- *saved in stable storage*

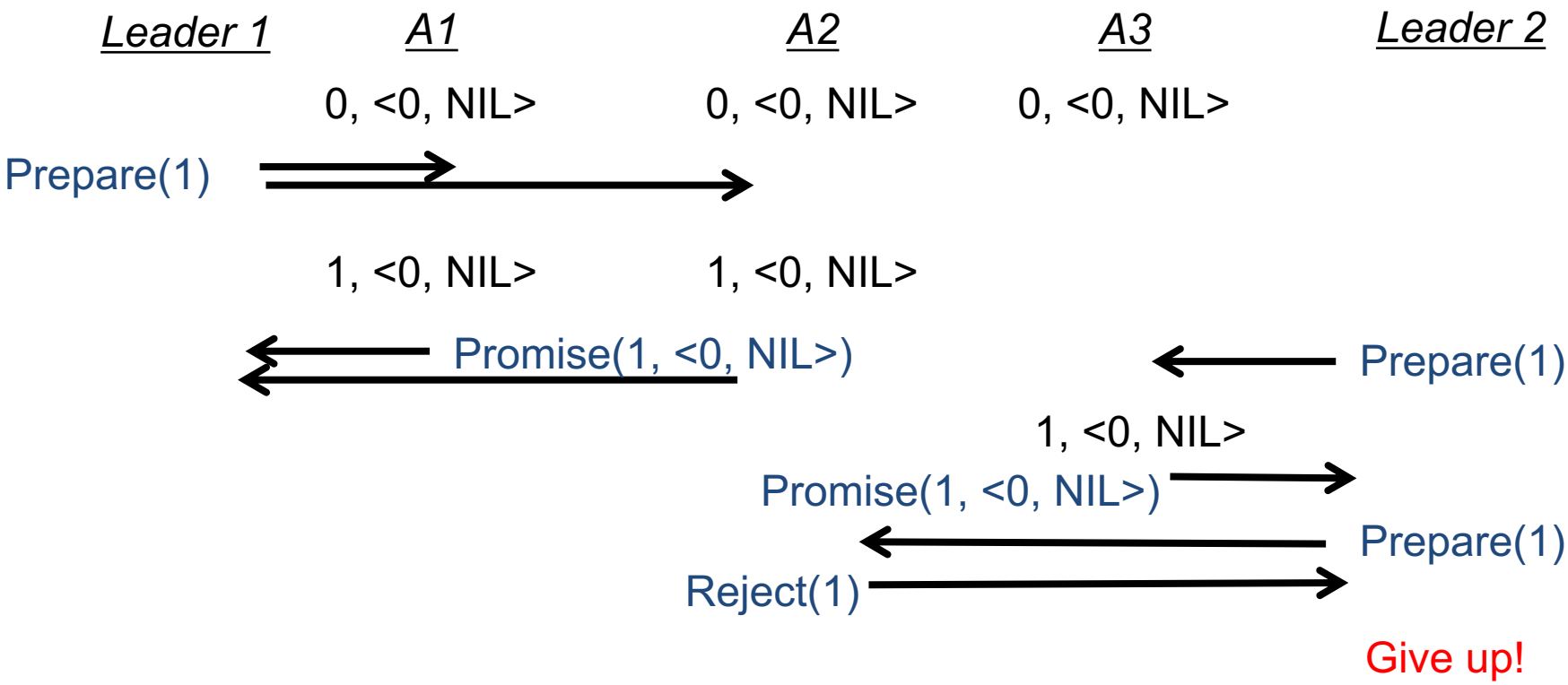
    return Accepted(<n, v>)

  else return Reject(n)

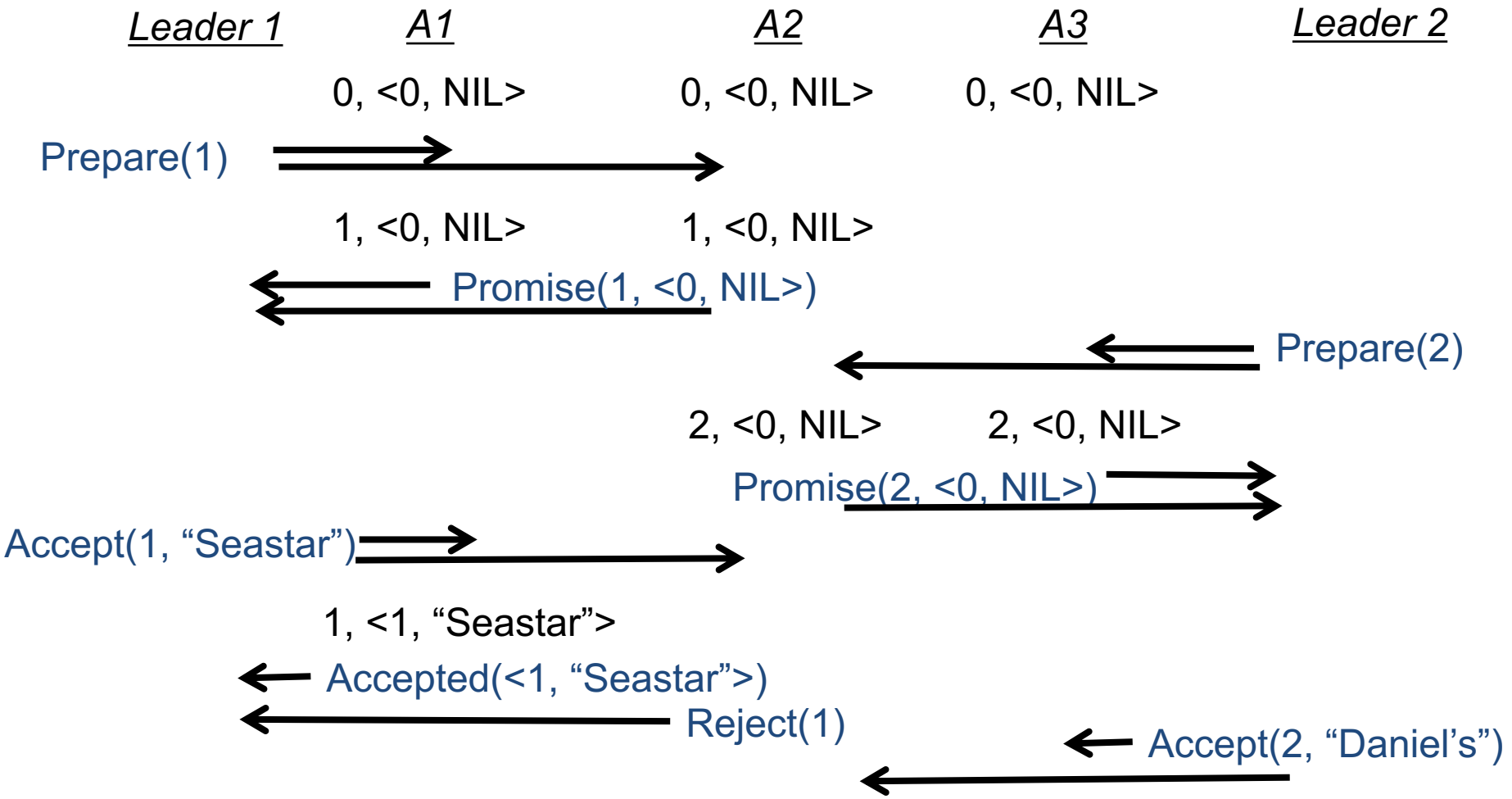
Paxos Example



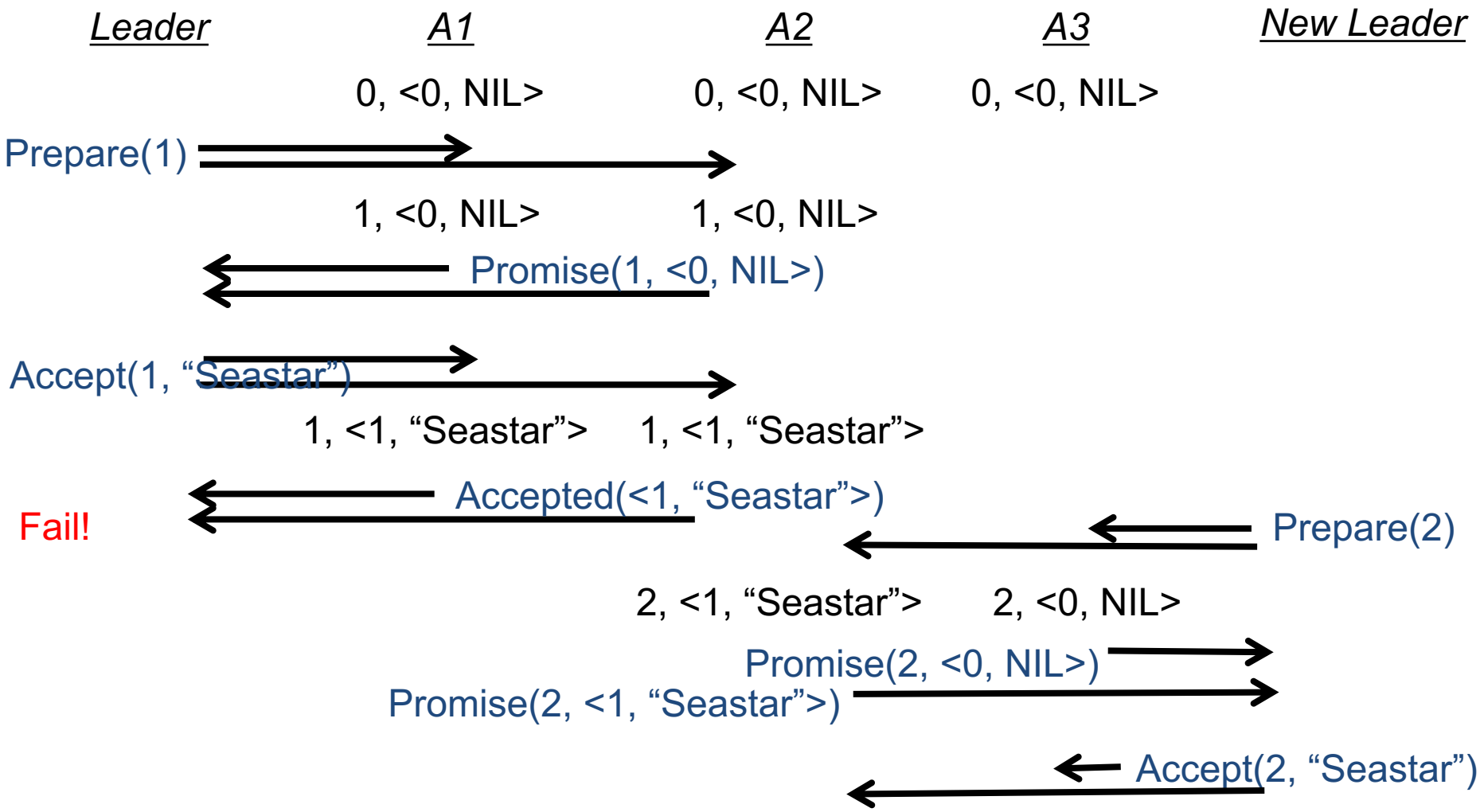
Paxos Example with Multiple Leaders



Another Paxos Example with Multiple Leaders



Paxos Example with Failed Leader



Paxos

Observations:

- Single proposer is desired for liveness, but not required for correctness
  - concurrent proposers may continually make new proposals with higher numbers before anyone can complete phase II, thereby preventing consensus
  - use “non-safe” election algorithm
- Proposers can timeout and retry
  - use exponential backoff

Paxos

- When is a value chosen?
  - When it has been accepted by a majority of acceptors
- Can multiple proposals be chosen?
  - Yes, but they all will choose the same value
- What if an acceptor crashes after accepting a proposal but before informing anyone?
  - Proposer can make a higher numbered proposal that will be accepted with the same value

Paxos

Optimizations:

- Prepare and Accept messages need only be sent to a majority of acceptors
  - need not be same majority in phase I and II
- Phase I can be performed before there are any values to propose
- If reaching consensus on a sequence of values, can batch phase I

Paxos Fault-Tolerance

To tolerate *f* faulty nodes:

- Requires 2*f*+1 acceptors
  - since need a majority to accept
- Requires *f*+1 proposers
  - since need one proposer to make a proposal
- Requires no learners

Applications of Paxos

Lots of uses and variations...

- Replicated state machines
  - agree on order of operations
  - proposed value = <seq #, op>
- Chubby lock service
- Group membership
- Transaction commit
- etc.

Variations on Paxos

- Multi-Paxos
- Flexible Paxos
- W-Paxos
- E-Paxos
- Etc.

Multi-Paxos
aka *Multiple-Decree Paxos*

- Agree on a sequence of values in a log

- Run instance of Paxos for each log position
- Commonly: Each value is an operation
  - Learners apply operations in order

Multi-Paxos
aka *Multiple-Decree Paxos*

Optimizations:

1. Use same leader for all instances
2. Run phase 1 for *all* slots in advance
3. Run phase 2 for each new value

Requires only one round of messages!

Raft

- Designed as a more understandable consensus protocol
- Really about log replication
- Published at USENIX ATC in 2014
- Several open-source implementations
- Used in database and other systems, e.g. TiDB, CockroachDB

Key Features of Raft

- ***Strong leader***
  - Leader issues ***all*** log updates
- ***Majority consenus***
  - Leader plus half followers for commit
- ***Leader election***
  - Uses ***randomized timers*** to elect leaders
- ***Membership changes***
  - New ***joint consensus*** approach

Raft Cluster

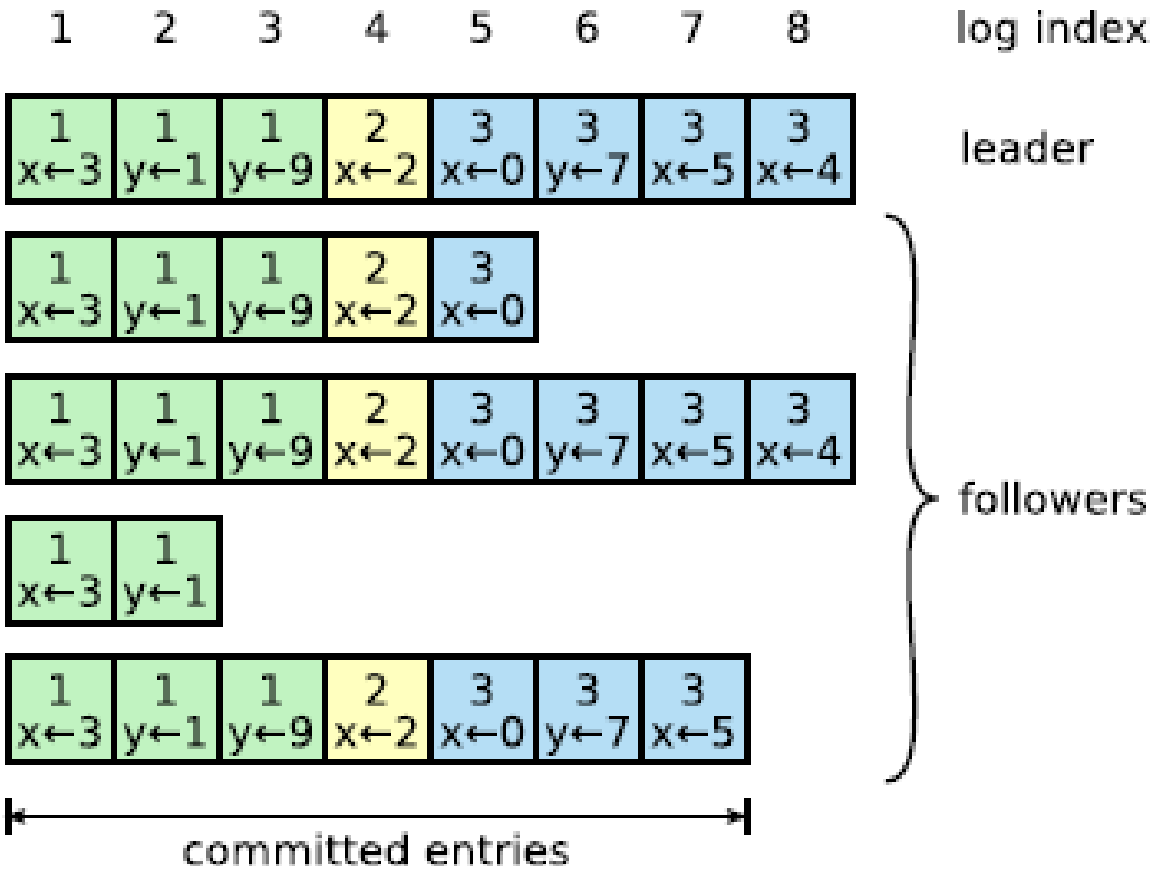
- Collection of servers
  - Often 5
- One *leader*
  - Accepts new write requests from clients
- Remaining *followers*
  - Accept requests only from leader
  - Initiate leader election if timeout

Raft Distributed Log

- Each server replicates log

- All logs contain same commands in same order
  - though some followers can store log prefix
  - no holes in log
- Append to the log in one round of messages

Raft Distributed Log



Part 2: Replicated State Machines

Fault-tolerance

- A system or algorithm is said to be *fault-tolerant* if in the presence of failures it:

  (1) continues to operate

  => high availability

  (2) produces correct results

  => high reliability

- A system is *n-fault-tolerant* if it tolerates *n* failures of type *t*
- Perhaps with reduced performance

Failures in Distributed Systems

Three major categories of failures:

- Processor failures
  - e.g. crashes
- Communication failures
  - lost, reordered, modified, or late messages
  - network partitions
- Storage failures
  - e.g. damaged disks

Classes of Processor Failures

Two major classes of faulty processors:

- *Fail-stop processor*
  - stops sending messages when crash occurs
  - persistent storage left uncorrupted
  - assumed by most systems
- *Byzantine processor*
  - behaves arbitrarily
  - can lie, cheat, … have bugs

Fail-stop Processor Failures

What can go wrong if a machine crashes while running a program?

- The program's work may be lost
- The program is unavailable
- The user may not be sure what state the program was in when it crashed
- The program's stable storage may be left in an inconsistent state

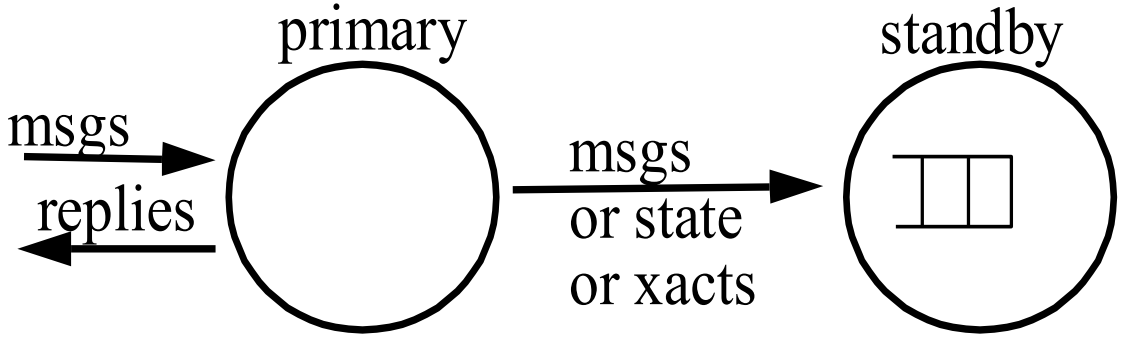
Highly-available Services

Two approaches for fault-tolerance:

- Primary and standby nodes
  - primary executes computation
  - standby is ready to take over if primary fails
  - e.g. Tandem's process pairs [Bartlett 81]
- Replicated state machines
  - identical nodes execute in parallel on identical inputs [Schneider 86]

Primary and Standby Nodes

- Basic model:



- Primary must checkpoint its state or inputs (e.g msgs) to standby
- Standby takes over when primary crashes

Primary and Standby Nodes

- Standby can be *hot* or *cold*
  - cold => do nothing until failure; then repeat computation from message log
  - hot => standby replicates program state and execution
  - hybrid is possible in which standby lags behind

Primary and Standby nodes

During normal (failure-free) operation:

- When can the primary reply to requests?
  - requests must be logged before reply is sent
- When can the standby prune its log?
  - never if cold
  - unless primary sends checkpointed state
- Do replies need to be logged?
  - not if execution is deterministic
- How is failure of primary detected?
  - standby must monitor and take over

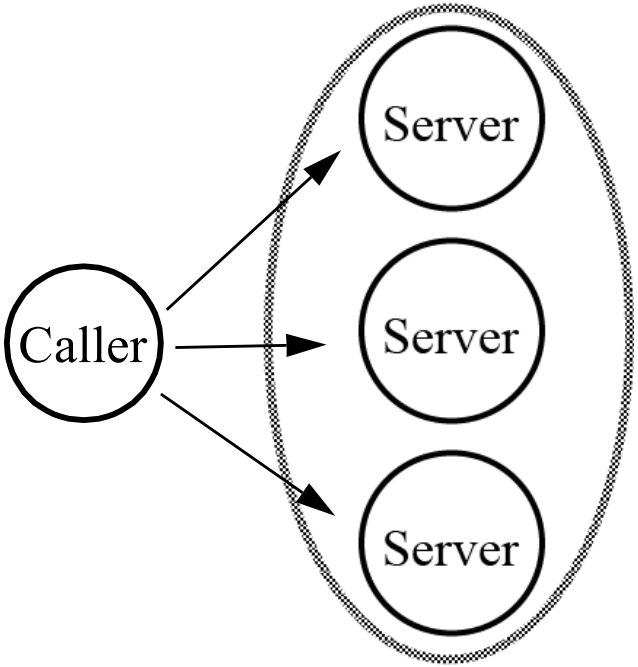
Primary and Standby nodes

When primary fails:

- Standby re-executes requests to reproduce primary's state
  - starts with latest checkpoint
  - assumes deterministic execution
  - but standby avoids external actions already taken by primary, e.g. sending messages
- What if reply is lost due to primary failure?
  - client will retransmit and standby will reply
- What about network partitions?

Replicated Computations

- Model for replicated state machine:



- Example: 3-modular redundancy

Replicated Computations

- Each server executes as a *state machine* [Lamport 84]
  - takes input, computes, changes internal state, produces results
- Assumptions?
  - Computation must be deterministic
  - Communication must be atomic and ordered
    - use atomic, totally ordered multicast
    - or use Paxos to reach agreement on next input
- Note: servers need not operate in lock step

Replicated State Machine with Paxos

- Servers agree on sequence of operations
- Propose <seq#, op>
- Want single proposer for liveness
  - elect leader to which clients send operations
- Phase I can be performed before there are any values to propose
  - can batch phase I for sequence
- Does not ensure atomicity

Replicated State Machine

On the client side…

- Callers must handle multiple results
- If fail-stop servers?
  - take first result
  - treat others as duplicates
- If Byzantine servers?
  - vote on results
  - need *2f+1* servers if *f* are faulty

Replicated State Machine

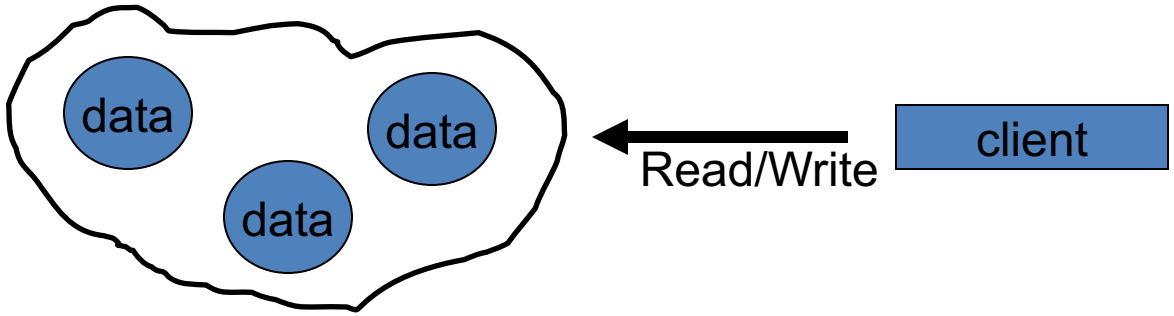
Recovery issues:

- Must deal with the failure of nodes and their subsequent reintegration
- Callers must be aware of changes to server group
  - e.g. must detect failures
- New nodes must be brought up-to-date
- This is non-trivial!

Part 3: Replicated Data

Replicated Data

- *Replicas* = copies of persistent data stored at multiple sites



- Support usual read/write operations
  - or application-specific operations
- Examples: distributed name servers, replicated file servers, clustered web sites, databases, content distribution networks

Replicated Data

Why replicate data?

- Increase availability
  - want data accessible even if machines crash
  - want to support disconnected operation
- Increase performance
  - locate data close to where it is used
- Other reasons: facilitate sharing, security, autonomy, system management, etc.

Replicated Data Approach

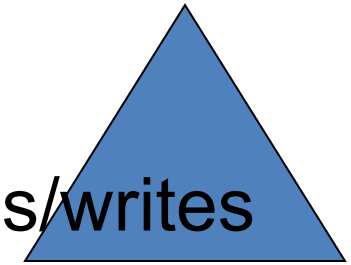
- Can use Paxos or Raft to replicate log
- Commands in log are database writes
- Writes are applied to database replica at each server in the logged order
  - servers eventually converge to consistent replicas
- What about database reads/queries?

Key challenge: Read consistency

Alternative CAP Theorem

- Characteristics of replicated data mechanisms:

Performance of reads/writes

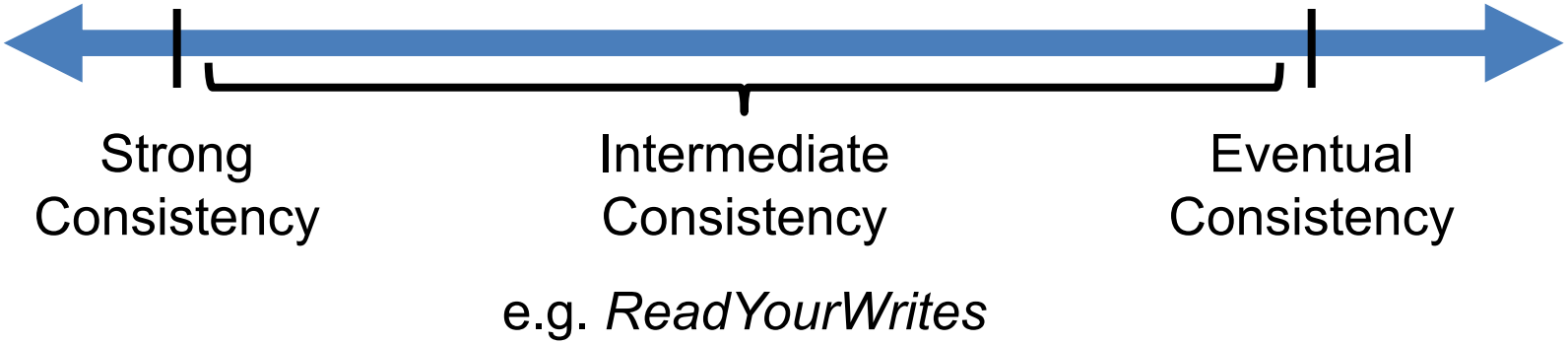


Availability of reads/writes

Consistency guarantees

- Cannot get best of all of these in one design

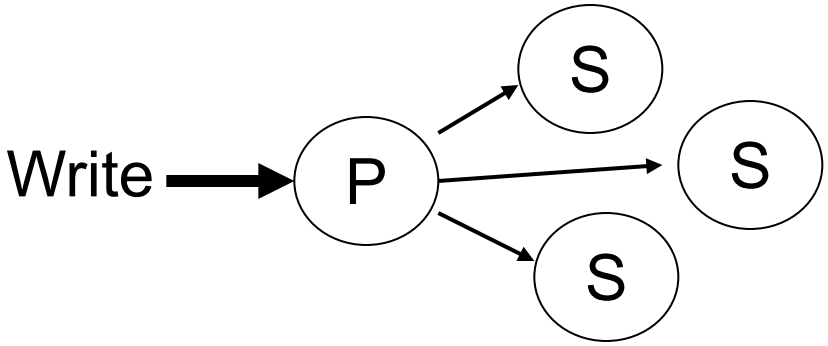
A Spectrum of Consistency



Lots of consistencies proposed in research community: probabilistic quorums, session guarantees, epsilon serializability, fork consistency, causal consistency, demarcation, continuous consistency, ...

Primary Copy

- Write to primary copy (e.g. Raft leader)
  - orders updates and detects/avoids conflicts

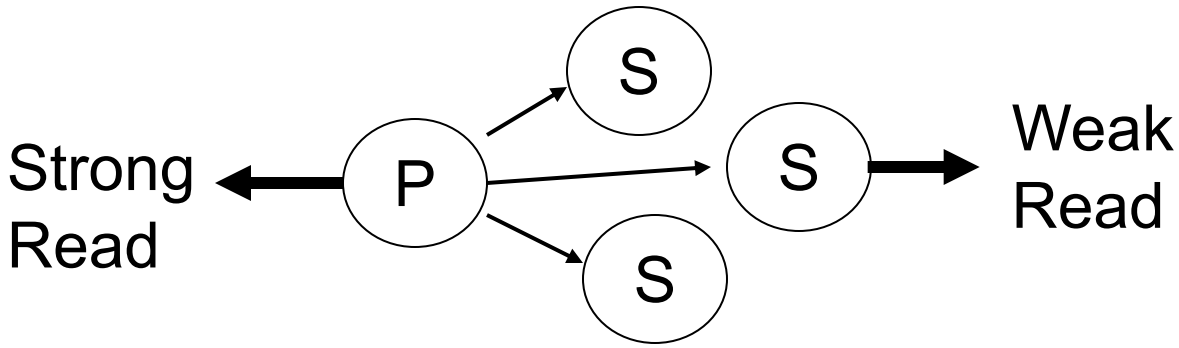


- Secondary copies contain snapshots
  - primary sends updates as soon as possible
  - or secondaries request updates periodically

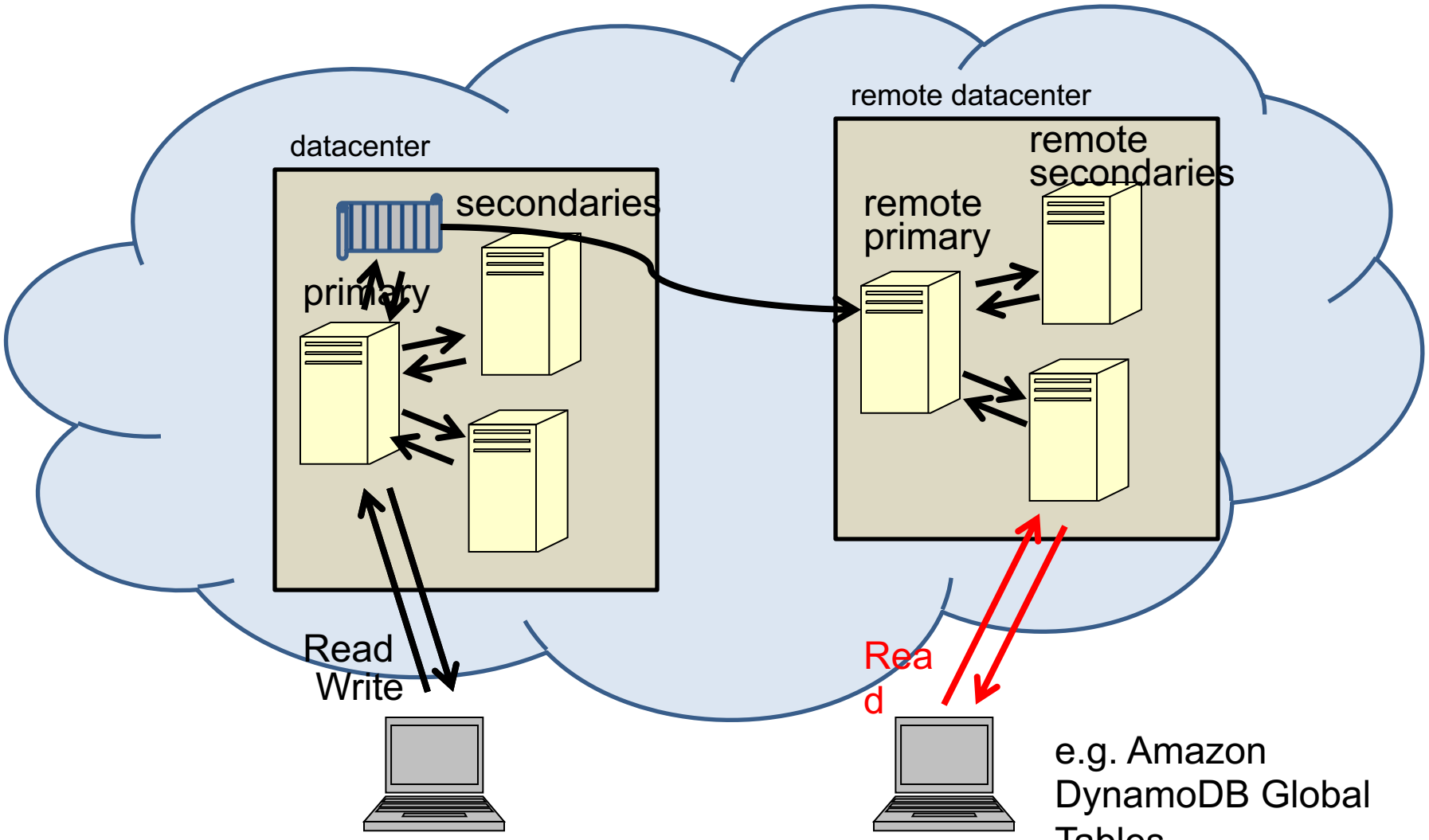
Primary Copy

What about Reads?

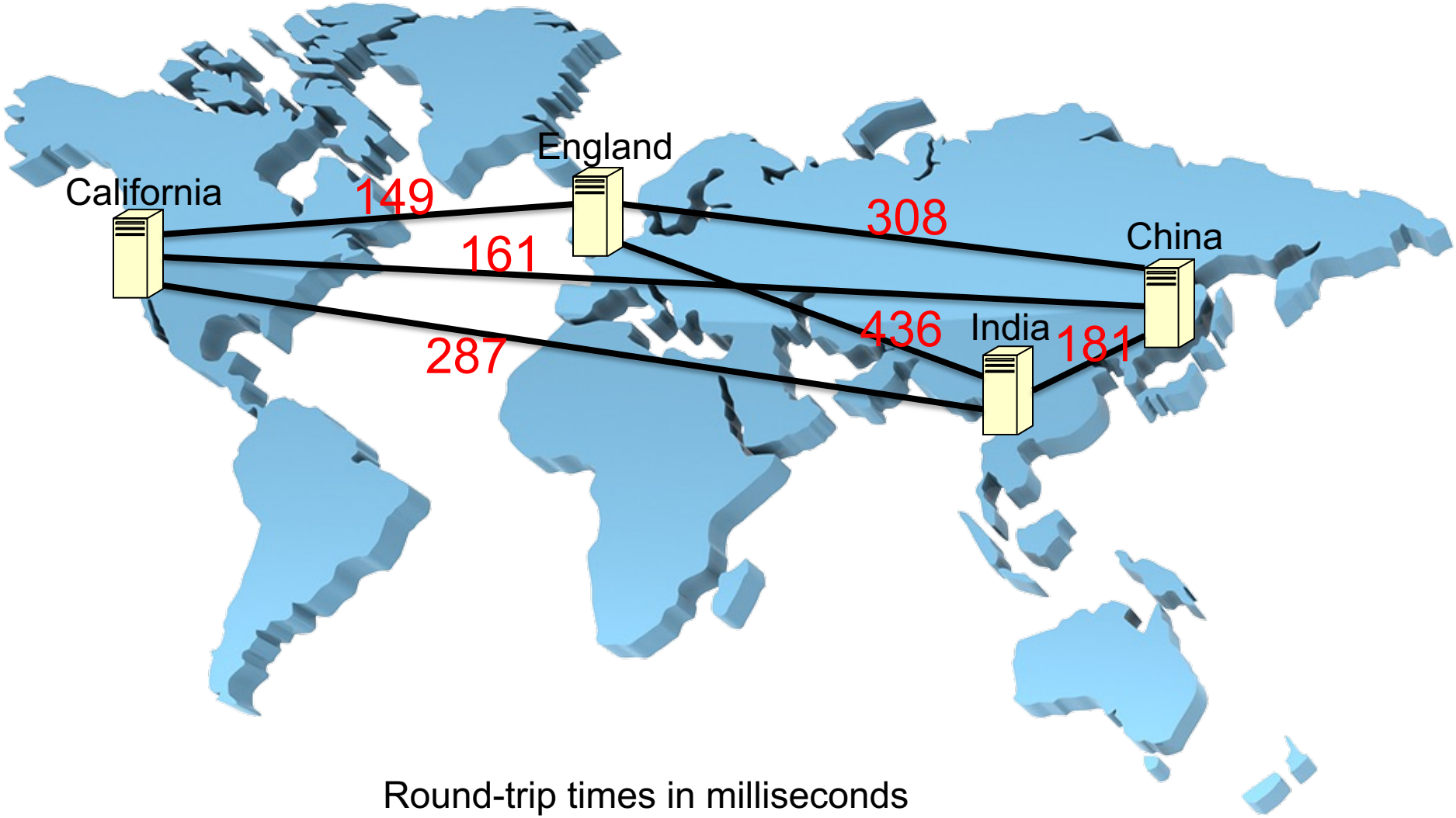
- If want strong consistency, read from primary
- If can tolerate eventual consistency, read any available copy



Geo-Replication in the Cloud



Geo-Replication Latency



Round-trip times in milliseconds

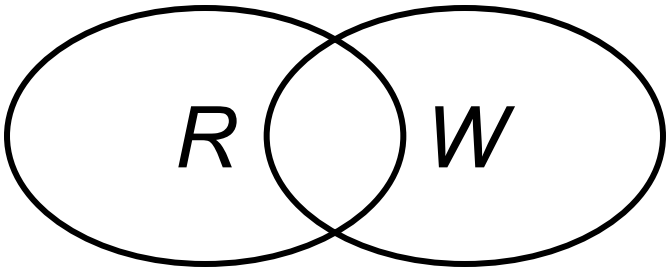
Majority Consensus (aka Quorums)

A read some/write some algorithm ...

- Must write *W* copies
  - *W* is write quorum
  - use transaction to atomically write W copies
- Must read *R* copies
  - *R* is read quorum
  - keep a version number for each copy
  - return copy with highest version number
- *R*=1, *W*=*N* is read any/write all

Majority Consensus

- To ensure strong consistency: $R + W > N$
  - N is total number of replicas
  - ensures that any read and write quorums overlap



- To ensure non-conflicting writes: $W > N/2$
- *Note*: Some NoSQL stores allow $R + W < N$

Majority Consensus

Observations:

- Some replicas may be out-of-date
  - not mutually consistent
- Can include any available sites in read or write quorum
- For reads, can retrieve *R* versions and then read from fastest up-to-date site
  - or read from all sites in parallel and choose highest version from first R responses

Majority Consensus

- For partial writes, must read to get current version before writing changes
  - must write whole file to out-of-date replicas
  - inefficient for append-only files

Majority Consensus

- Can adjust *R* and *W* to tradeoff availability and performance
- No recovery necessary for failed servers
- No need to detect network partitions
- Read and writes may have poor performance
- Data may be inaccessible in minority partition
  - nobody may be able to read or write data

Weighted Voting

Like majority consensus but...

- Can assign votes to replicas [Gifford 79]
  - each replica has some number of votes
  - one vote per replica = majority consensus
- *R* votes needed to read, *W* to write
- *V* is the total number of votes
- $R + W > V$
- $W > V/2$

Weighted Voting

Observations:

- High performance or highly-reliable sites can be given more votes
- Cached copies can be given zero votes
  - results in validate-before-use caching scheme
- Lots of variations, e.g. voting with witnesses
  - witness holds a version number but not the data
- Several papers on how to assign votes and adjust them dynamically as failures occur

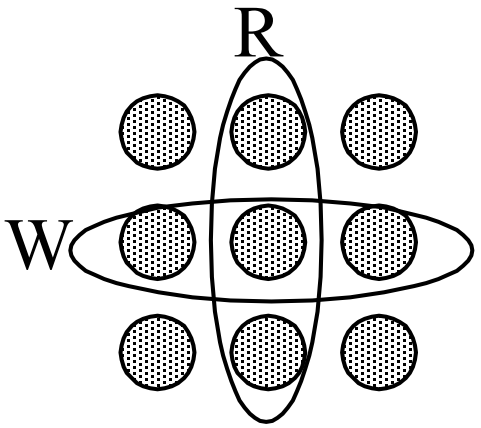
Structured Quorums

Also called voting with structures

- Organize servers in a grid or hierarchy
- Use this structure to restrict the choice of read and write quorums
- Can get smaller quorums, and hence better performance and availability, for the same number of servers

Structured Quorums

- Example: 9 replicas laid out in a 3x3 grid



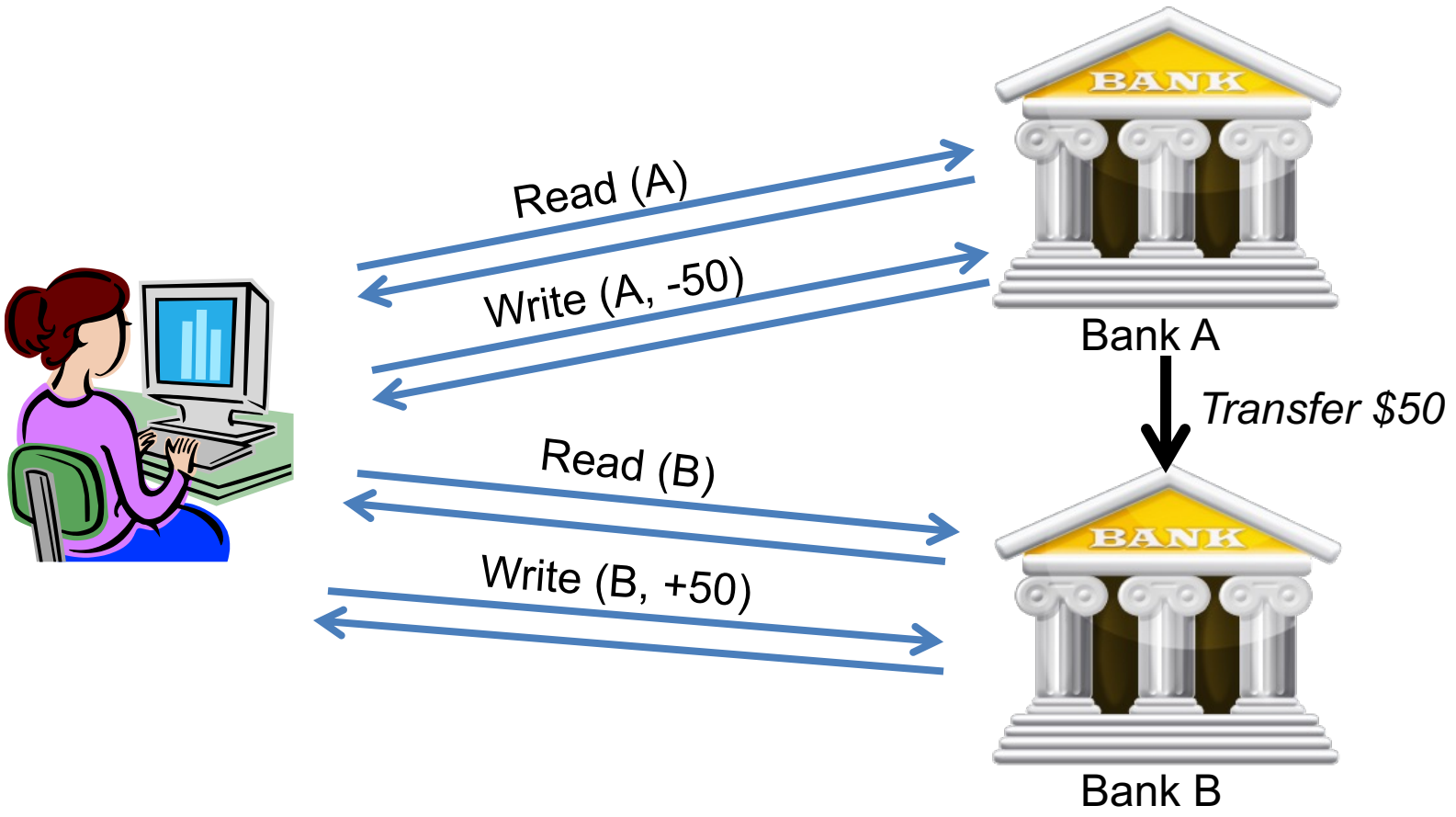
- Write a row, read a column => *W*=3, *R*=3
- Better than majority consensus?
  - *W*=5, *R*=5 or *W*=7, *R*=3

Part 4: Transaction Commit

Transactions

- *Transaction* = a set of operations with the following ACID properties:
  - Atomicity: all or none of the operations are performed,
  - Consistency: the operations correctly transform the computation state,
  - Isolation: transactions appear to execute in some order, also known as serializability,
  - Durability: committed updates are made to stable storage, also called permanence.

Example: Bank Transfer



Atomicity

- An operation is *atomic* if it is indivisible with respect to failures
  - i.e. all or nothing
- Atomic operations move computations from one consistent state to another
- Atomicity is key to fault-tolerance
  - e.g. update replicated data

Distributed Transactions

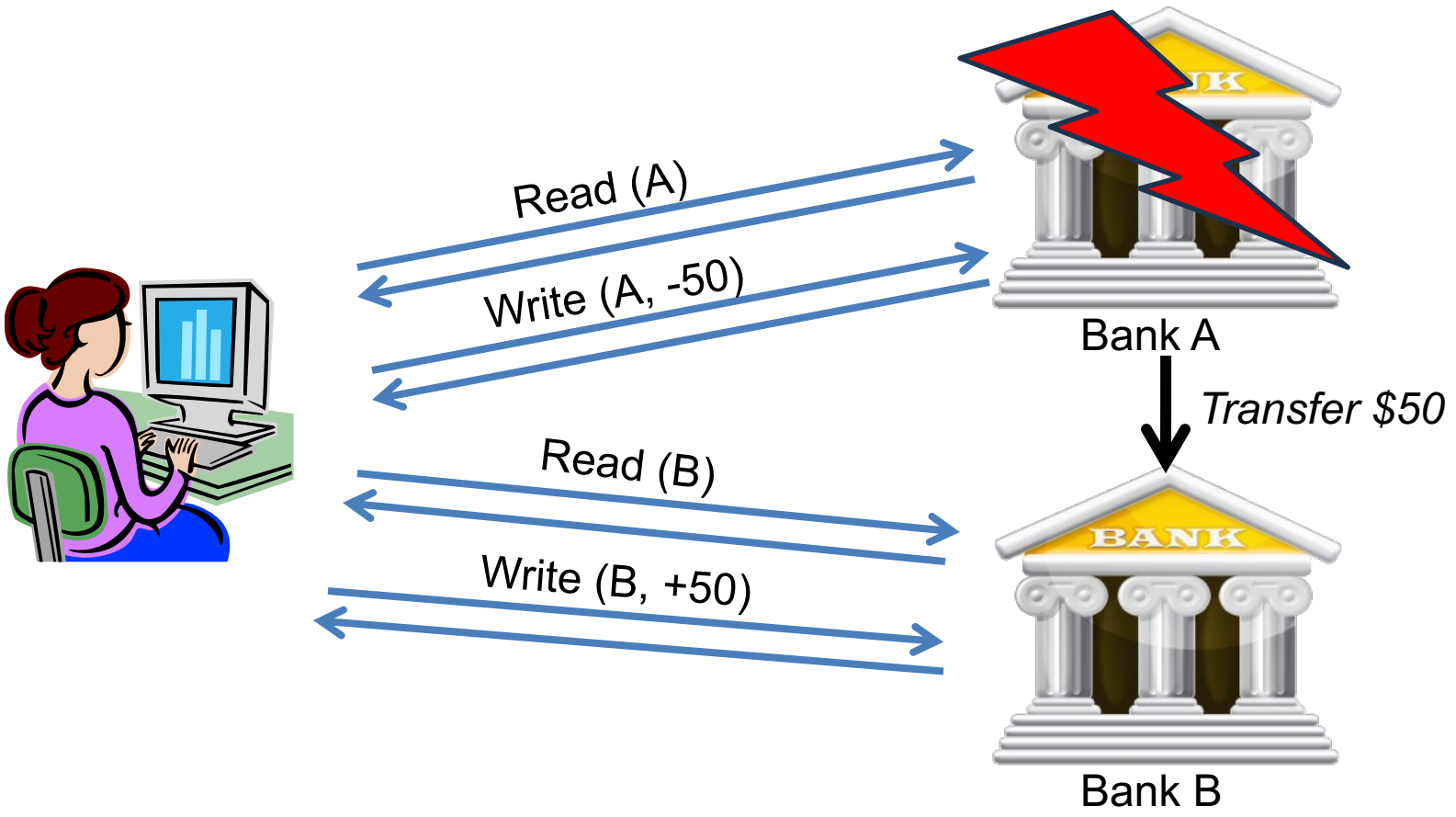
- Implementing a distributed transaction in two easy steps:

1. Run a transaction at each site
   - using conventional transaction mechanisms
2. All sites agree to either commit or abort their local transactions
   - using two-phase commitment protocol

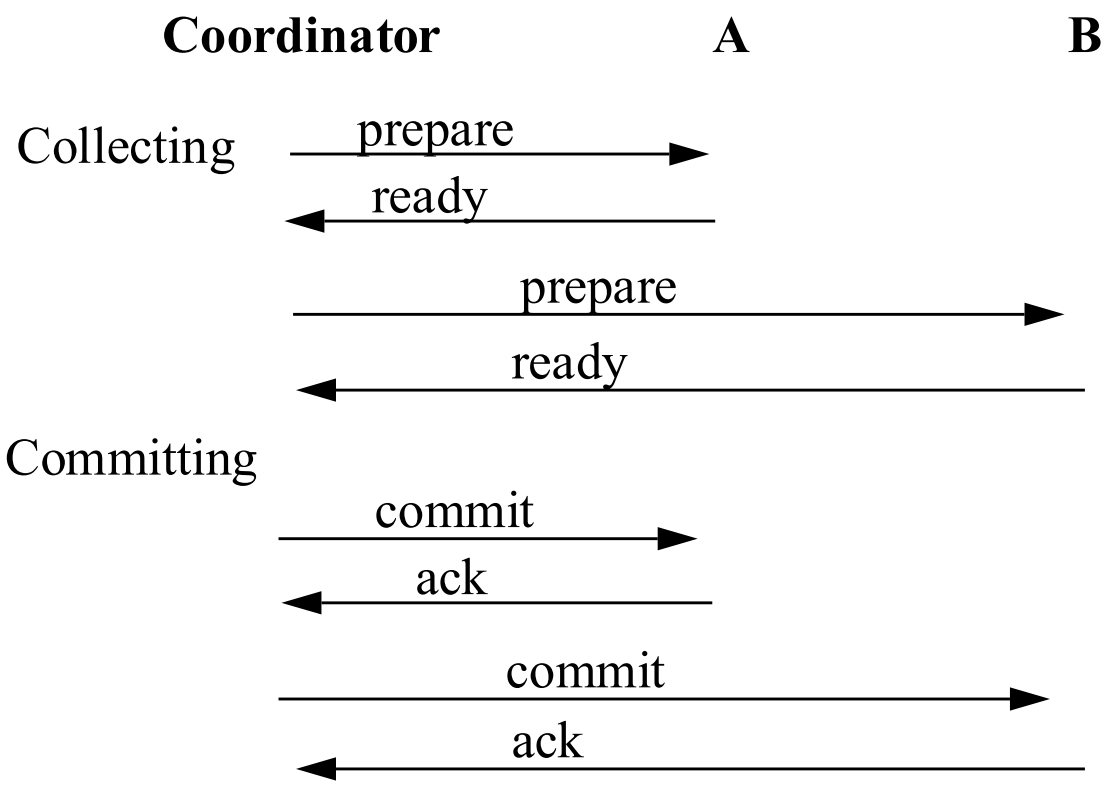
Committing Transactions

- Why not simply send a “commit” message to each site using consensus?
- Because need more than majority consensus
  - need all sites to either commit or abort
- Because consensus protocols only guarantee delivery to non-failed sites
- Because sites need may unilaterally abort
  - if conflicts arise or if they crash

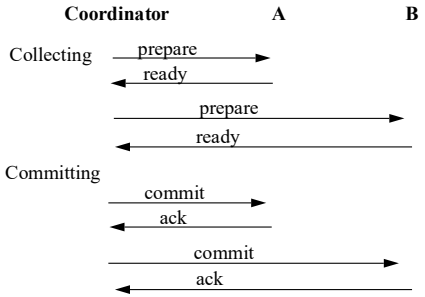
Example: Bank Transfer



Two-phase Commit



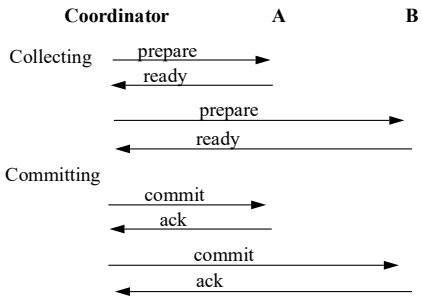
Two-phase Commit



Observations:

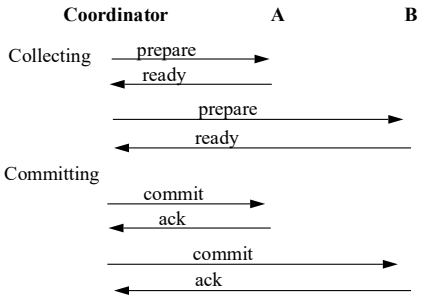
- Commit protocol runs at end of transaction, after all updates have been tentatively done
- Any site can unilaterally abort the transaction (until they respond with “ready” message)
- After first phase, any site must be willing and prepared to commit or abort
  - i.e. all updates must be recorded on stable storage
- Coordinator makes commit/abort decision

2PC: Site Failures



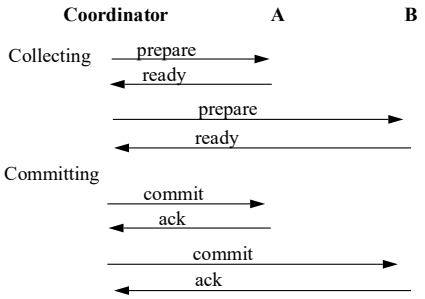
- What if a site fails before replying “ready”?
- Coordinator times out and aborts
  - no choice since failed site lost transaction state (most likely)
  - or site replies “no such transaction”
- What if a site fails after replying “ready”?
- On recovery, site must learn of transaction outcome
  - coordinator may have committed or aborted

2PC: Coordinator Failures



- What if coordinator fails before sending any "prepare" messages?
- Sites can still abort (and should)
  - can ping coordinator to see if failed
- What if coordinator fails after sending some but not all "prepare" messages?
- Some sites in "ready" phase and some not
  - ready sites can check other sites or wait
  - non-ready sites can abort

2PC: Coordinator Failures



- What if coordinator fails between first and second phases?
  - i.e. all sites have responded “ready” but no sites have received commit/abort decision
- Sites must wait for coordinator to recover
  - could ask other sites for outcome
  - generally, can’t tell if coordinator committed
  - i.e. there’s a window of vulnerability
  - three-phase commit reduces vulnerability

BASE Transactions

- BASE = Basically Available Soft-state Eventually consistent
- Used by EBay and others to avoid two-phase commit
- Use local transactions only
- Queue updates to remote sites
  - perform them asynchronously
  - and exactly once

BASE Example: Bank Transfer

Client code:

```
BeginTx ();
    QueueMsg (A, -50);
    QueueMsg (B, +50);
CommitTx ();
```

Bank B's code:

```
msg = DequeueMsg () at client;
BeginTx ();
    balance = Read (B);
    balance = balance + 50;
    Write (B, balance);
CommitTx ();
```

Bank A's code similar

BASE Example: Bank Transfer

Bank A's code:

```
BeginTx ();
    balance = Read (A);
    balance = balance - 50;
    Write (A, balance);
    QueueMsg (B, +50);
CommitTx ();
```

Bank B's code:

```
msg = DequeueMsg () at A;
BeginTx ();
    balance = Read (B);
    balance = balance + 50;
    Write (B, balance);
CommitTx ();
```

BASE Transactions

- Use reliable messaging, e.g. Kafka
- Or Multi-Paxos/Raft
- All participants see all sub-transactions in shared log
- Each participant *eventually* performs its own updates locally

BASE Transactions

Problem:

- Must perform local update atomically and *exactly once*

Solution:

- CommitDB stores sequence number of last performed operation

BASE Example: Bank Transfer

Bank B's code:

```
slot = Select from CommitDB;
Read operation in slot = (B, +50);
BeginTx ();
        balance = Read (B);
        balance = balance + 50;
        Write (B, balance);
        Insert slot+1 into CommitDB;
 CommitTx ();
```

BASE (vs. ACID)

Pros:

- only local transactions
- high availability
- good performance

Cons:

- eventual consistency
- no isolation/conflict resolution

Part 5: Conclusions and Questions

Lessons from Today

- Consensus algorithms are fundamental building blocks in distributed systems
- Used for replicated state machines
- Used for replicated databases
- Similar but not identical protocols used for transaction commit